Algoritmia

Práctica 2

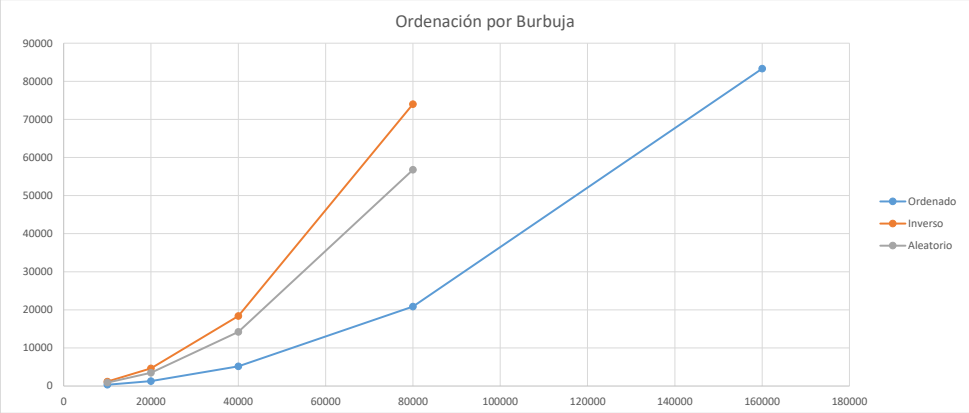
21/2/2026

Contenidos

1. Ordenación por Burbuja 2
2. Ordenación por Selección 3
3. Ordenación por Inserción 4
4. Ordenación por *QuickSort* 6
5. Tiempo teórico de ordenación de un vector 7
6. Mediciones con tiempos bajos 9
7. Condiciones de medida 11

1. Ordenación por Burbuja

n	T_{ord} (ms)	T_{inv} (ms)	T_{alea} (ms)	*Repeticiones*
10.000	336	1.176	900	1
20.000	1.309	4.642	3.500	1
40.000	5.156	18.309	14.217	1
80.000	20.873	74.005	56.764	1
160.000	83.314	FdT	FdT	1
320.000	FdT	FdT	FdT	1



Para este algoritmo; los casos mejor, peor y medio tienen todos complejidad teórica de $O(n^2)$. Veamos si los datos empíricos lo respaldan:

n	n^2/T_{ord}	n^2/T_{inv}	n^2/T_{alea}
10.000	297.619,0476	85.034,0136	111.111,1111
20.000	305.576,7762	86.169,7544	114.285,7143
40.000	310318,0760	87.003,8064	112.541,3238
80.000	306.616,2027	86.480,6432	112.747,5160
160.000	307.271,2869	-	-

Los cocientes son estables, se respalda la complejidad teórica.

2. Ordenación por Selección

n	T_{ord} (ms)	T_{inv} (ms)	T_{alea} (ms)	*Repeticiones*
10.000	312	302	324	1
20.000	1.257	1.139	1.265	1
40.000	5.014	4.694	5.076	1
80.000	20.133	18.684	20.003	1
160.000	81.081	76.106	80.194	1
320.000	FdT	FdT	FdT	1

Este algoritmo no depende tanto del orden previo del vector

n	n^2/T_{ord}	n^2/T_{inv}	n^2/T_{alea}
10.000	320.512,8205	331.125,8278	308.641,9753
20.000	318.217,9793	351.185,2502	316.205,5336
40.000	319.106,5018	340.860,6732	315.208,8258
80.000	317.886,0577	342.539,0709	319.952,0072
160.000	315.733,6491	336.372,9535	319.225,8772

Los cocientes son estables en los tres casos: se sigue la complejidad teórica

3. Ordenación por Inserción

Este algoritmo presenta complejidades teóricas distintas según el orden previo del vector.

Inserción con vector previo ordenado:

n	Tiempo total (ms)	Repeticiones	n/T_{total}
10.000	223	1000	44,8430
20.000	406	1000	49,2611
40.000	831	1000	48,1348
80.000	1.634	1000	48,9596
160.000	3.239	1000	49,3980
320.000	6.500	1000	49,2308
640.000	12.972	1000	49,3370
1.280.000	26.190	1000	48,8736

Los datos empíricos respaldan la complejidad teórica de $O(n)$

Resto de casos:

n	Repeticiones	T_inverso (ms)	T_aleatorio (ms)	n^2/T_{inv}	n^2/T_{alea}
10.000	1	324	174	308.641,975	574.712,644
20.000	1	1.315	714	304.182,510	560.224,090
40.000	1	5.284	2.697	302.800,908	593.251,761
80.000	1	21.431	10.942	298.632,822	584.902,212
160.000	1	85.710	43.823	298.681,601	584.168,131
320.000	1	Fdt	174.247	-	587.671,524

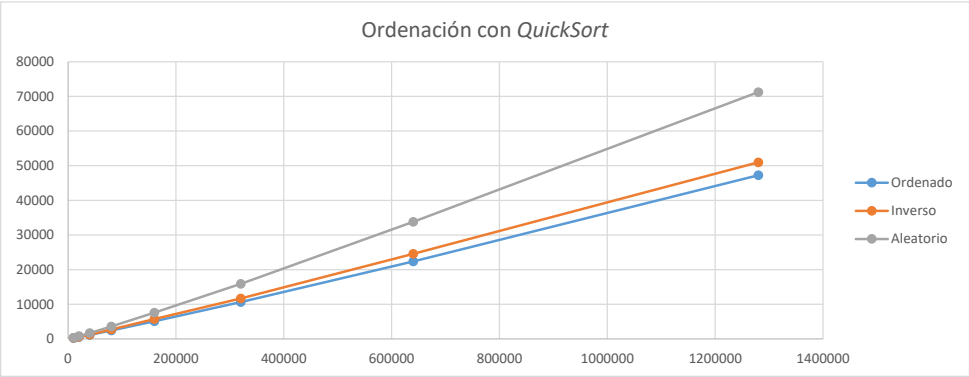
De nuevo, los datos respaldan la complejidad teórica de $O(n^2)$

Se han omitido valores para evitar la distorsión de la gráfica

Podemos observar una clara diferencia entre los tiempos empíricos incluso representando únicamente los valores de n para los que no se va fuera de tiempo el algoritmo en los casos inverso y aleatorio.

4. Ordenación por *QuickSort*

n	T_{ord} (ms)	T_{inv} (ms)	T_{alea} (ms)	*Repeticiones*
10.000	248	297	377	1
20.000	542	607	794	1
40.000	1.146	1.245	1.699	1
80.000	2.447	2.718	3.614	1
160.000	5.083	5.725	7.576	1
320.000	10.653	11.701	15.895	1
640.000	22.383	24.549	33.809	1
1.280.000	47.247	50.977	71.247	1



El algoritmo con mejor rendimiento entre los que hemos visto hasta ahora

Podemos observar en la gráfica que los tiempos siguen claramente un patrón de líneas rectas. Al estar ambos ejes en escala logarítmica, este es un fuerte indicativo de que el algoritmo implementado sigue su complejidad teórica de $O(n \log n)$.

Nota: no se alcanza el peor caso de complejidad teórica $O(n^2)$ debido a los casos de prueba abordados y al haber tomado el elemento central como pivote en nuestra implementación.

n	$n\log(n)/T_{ord}$	$n\log(n)/T_{inv}$	$n\log(n)/T_{alea}$
10.000	161,2903	134,6801	106,1008
20.000	158,7096	141,7143	108,3383
40.000	160,6304	147,8573	108,3475
80.000	160,2972	144,3146	108,5355
160.000	163,8126	145,4427	109,9075
320.000	165,3664	150,5553	110,8303
640.000	166,0169	151,3689	109,9102
1.280.000	165,4545	153,3482	109,7201

Los cocientes son estables en los tres casos.

5. Tiempo teórico de ordenación de un vector

En el problema propuesto, el vector está ordenado aleatoriamente. Por tanto, hemos de usar la complejidad teórica correspondiente al caso medio para cada algoritmo. Esta es de $O(n^2)$ para todos los algoritmos vistos; a excepción de *QuickSort*, que tiene una complejidad de $O(n \log n)$ para este caso.

Sea f_i la función de complejidad teórica para el algoritmo teórico i-ésimo, conocido el tiempo t_1 que emplea ese mismo algoritmo para una carga n_1, podemos predecir el tiempo para otra carga n_2 como:

$$t_2 = t_1 \cdot \frac{f_i(n_2)}{f_i(n_1)}$$

Intentaremos resolver este problema teórico usando el tiempo de mayor carga medido para cada algoritmo. Al usar el caso de mayor n estudiado, las variaciones que pueden ocurrir al llamar a una función ajenas al algoritmo de ordenación se ven mitigadas: El coste fijo del problema (ajeno al tamaño de la carga) se vuelve despreciable.

- **Burbuja:** 56.764ms para una repetición en el caso $n = 80.000$.

$$56.764ms \cdot \frac{81.920.000^2}{80.000^2} \cdot \frac{1d}{86.400.00ms} = 688,9047d$$

- **Selección:** 80.194ms para el caso $n = 160.000$

$$80.194ms \cdot \frac{81.920.000^2}{160.000^2} \cdot \frac{1d}{86.400.00ms} = 243,3145d$$

- **Inserción:** 174.247ms para el caso $n = 320.000$

$$174.247ms \cdot \frac{81.920.000^2}{320.000^2} \cdot \frac{1d}{86.400.00ms} = 132,1696d$$

- ***QuickSort:*** 71.247ms para el caso $n = 1.280.000$ en 100 repeticiones.

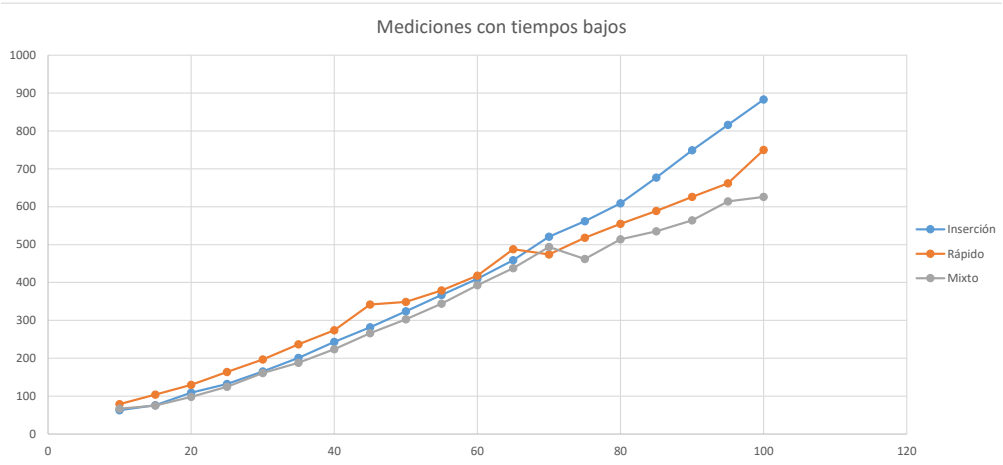
$$\frac{71.247ms}{100} \cdot \frac{81.920.000 \cdot \log(81.920.000)}{1.280.000 \cdot \log(1.280.000)} \cdot = 59,0835ms$$

  En días, son $6,8384 \cdot 10^{-7}d$.

6. Mediciones con tiempos bajos

En primer lugar, hemos medido los tiempos de Inserción y *QuickSort* para cargas bajas. Para ello, hemos tenido en cuenta el tiempo del bucle al completo, en lugar de ir sumando los tiempos de cada paso. De este modo, no tenemos problemas con hipotéticas medidas nulas.

Aunque *QuickSort* maneja mejor problemas grades, es más lento que Inserción para cargas bajas. Tras medir estos dos algoritmos empíricamente, hemos diseñado un tercero que usa el más rápido según el tamaño del problema: Divide la carga con *QuickSort* hasta llegar a un tamaño en el que Inserción es más rápido, en cuyo caso llama a este último método. Por simplicidad, se presentan los tiempos de los tres algoritmos en una misma tabla:



n	Repeticiones	T_{ins} (ms)	T_{quick} (ms)	T_{mixto} (ms)
10	10.000	63	79	67
15	10.000	76	104	75
20	10.000	109	130	98
25	10.000	132	164	125
30	10.000	165	197	161
35	10.000	201	237	188
40	10.000	243	274	224
45	10.000	282	342	266
50	10.000	324	349	303
55	10.000	367	379	344
60	10.000	410	418	393
65	10.000	459	488	438
70	10.000	521	474	494
75	10.000	562	518	462
80	10.000	609	555	514
85	10.000	677	589	535
90	10.000	749	626	564
95	10.000	816	662	614
100	10.000	883	750	626

El nuevo algoritmo mixto parece dar buenos resultados.

7. Condiciones de medida

Todas las tomas de tiempos de esta memoria han sido realizadas en Eclipse IDE usando el argumento de máquina virtual `-Xint` para la ejecución de los programas de cronometrado.

Los apartados 1 al 4 de esta memoria han sido medidos con el siguiente equipo:
`CPU 12th Gen Intel(R) Core(TM) i5-12400 @ 2,5GHz + 16GB RAM`

El apartado 6 ha sido medido usando el siguiente hardware:
`CPU: Intel Core i7-6700HQ CPU @ 2,60GHz + 16GB RAM`